1. Изучение команды find

План:

1. Создать каталог и в нем несколько файлов
2. С использованием команды touch установить для созданных файлов различное время последнего доступа
3. С использованием find найти и переместить в указанный каталог, например ./old, файлы, к которым не было обращений более 14 дней
4. Каталог ./old исключить из поиска

С помощью команды mkdir создается директория, с помощью команды touch создаются файлы, либо если файлы с таким названием уже есть, меняется время последнего доступа.

```
User@LAPTOP-ORANGE MINGW64 ~/downloads
$ mkdir OS_1

User@LAPTOP-ORANGE MINGW64 ~/downloads
$ cd OS_1

User@LAPTOP-ORANGE MINGW64 ~/downloads/OS_1
$ touch file1 file2 file3

User@LAPTOP-ORANGE MINGW64 ~/downloads/OS_1
$ ls
file1  file2  file3

User@LAPTOP-ORANGE MINGW64 ~/downloads/OS_1
$ touch file1 -d 20260927

User@LAPTOP-ORANGE MINGW64 ~/downloads/OS_1
$ touch file2 -d 20260926

User@LAPTOP-ORANGE MINGW64 ~/downloads/OS_1
$ touch file3 -d '14 days ago'
```

С помощью команды find находим файлы, доступ к которым был больше 14 дней назад и перемещаем их в папку old.

```
User@LAPTOP-ORANGE MINGW64 ~/downloads/OS_1
$ find -mtime +14
./file2

User@LAPTOP-ORANGE MINGW64 ~/downloads/OS_1
$ find -mtime +14 -exec mv {} ./old \;
mv: './old/file2' and './old/file2' are the same file

User@LAPTOP-ORANGE MINGW64 ~/downloads/OS_1
$ find . -mtime +14 -exec mv {} ./old \;
mv: './old/file2' and './old/file2' are the same file

User@LAPTOP-ORANGE MINGW64 ~/downloads/OS_1
$ find
.
./file1
./file3
./old
./old/file2

User@LAPTOP-ORANGE MINGW64 ~/downloads/OS_1
$ find -type f
./file1
./file3
./old/file2
```

Папку исключаем путем отсекания пути в неё.

```
User@LAPTOP-ORANGE MINGW64 ~/downloads/OS_1
$ find -path './old' -prune -o -print
.
./file1
./file3
```

2. Изучение сценариев bash

1. Вывод всех аргументов сценария в обратном порядке:

```
  GNU nano 8.5                                script1
#!/bin/bash

i=$#
while [ $i -gt 0 ]
do
        echo ${!i}
        i=$(expr $i - 1)
done
```

2. Создание файла, если его не существует, и установка прав доступа к нему:
Проверяем на наличие аргументов, затем проверяем, существует ли файл с названием из аргумента ”$1”, отдельно проверяем, есть ли второй аргумент, и если есть — используем команду chmod.

```
  GNU nano 8.5                          mkfile
#!/bin/bash

if [ $# -eq 0 ]
then
        echo "Ошибка: Имя файла не указано"
        exit 1
fi

if [ ! -e "$1" ]
then
        if touch "$1"
        then
                echo "Файл успешно создан"
        else
                echo "Не удалось создать файл"
                exit 2
        fi
else
        echo "Файл с таким названием уже существует"
fi

if [ "$2" ]
then
        if chmod "$2" "$1"
        then
                echo "Права доступа успешно установлены"
        else
                echo "Не удалось установить права доступа"
                exit 3
        fi
fi
```

3. Калькулятор для арифметических действий (+, -, *, /):



3. Изучение сценариев bash (часть 2)

План:

1. Написать скрипт ptxt, который выводит на экран заданный текст заданное количество раз через заданный интервал времени. Предусмотреть следующий синтаксис вызова: «ptxt -n <число выводов текста> -t <таймаут вывода> -- <текст для вывода>». Использовать команды case и shift для получения параметров
2. Написать скрипт ptxt как в задании 1, но использовать getopts
3. Написать скрипт, который сортирует и выводит переданные в него аргументы в алфавитном порядке.

Так как 1 и 2 задание требуют один и тот же скрипт в разных реализациях, я создал 2 реализации с названиями ptxt1 и ptxt2 соответственно.

1. Скрипт ptxt, который выводит на экран заданный текст заданное количество раз через заданный интервал времени:

```
  GNU nano 8.5                          ptxt1
#!/bin/bash

while [ -n "$1" ]
do
        case "$1" in
        -n)
                n="$2"
                shift 2
                ;;
        -t)
                t="$2"
                shift 2
                ;;
        --)
                shift
                text="$*"
                if [ -z "$text" ]
                then
                        echo "Введите текст для вывода"
                fi
                break
                ;;
        *)
                echo "Передайте аргументы в формате "\
"'-n <число выводов текста> -t <таймаут вывода> -- <текст>'"
                exit 1
                ;;
        esac
done

if [ -z "$n" -o "$n" -le 0 ]
then
        n=1
fi
if [ -z "$t" -o "$t" -lt 0 ]
then
        t=0
fi

while [ "$n" -ge 1 ]
do
        echo "$text"
        n=$(expr $n - 1)
        if [ "$n" -ge 1 ]
        then
                sleep "$t"
        fi
done
```

2. Скрипт ptxt с использованием getopts:

```
  GNU nano 8.5                          ptxt2
#!/bin/bash

while getopts n:t: a
do
        case "$a" in
        n)
                n="$OPTARG"
                ;;
        t)
                t="$OPTARG"
                ;;
        \?)
                echo "Передайте аргументы в формате "\
"'-n <число выводов текста> -t <таймаут вывода> -- <текст>'"
                exit 1
                ;;
        esac
done

shift $((OPTIND - 1))
text="$*"

if [ -z "$text" ]
then
        echo "Передайте текст для вывода в аргументы"
        exit 2
fi

if [ -z "$n" ] || [ "$n" -le 0 ]
then
        n=1
fi
if [ -z "$t" ] || [ "$t" -lt 0 ]
then
        t=0
fi

while [ "$n" -ge 1 ]
do
        echo "$text"
        n=$(expr $n - 1)
        if [ "$n" -ge 1 ]
        then
                sleep "$t"
        fi
done
```

3. Скрипт, который сортирует и выводит переданные в него аргументы в алфавитном порядке (используется сортировка методом пузырька):

```
  GNU nano 8.5                          sorter
#!/bin/bash

if [ $# -eq 0 ]
then
        echo "Передайте аргументы"
        exit 1
fi

arr=("$@")
size=${#arr[@]}

for ((i = 0; i < size; i++)); do
        for ((j = 0; j < size - i - 1; j++)); do
                if [[ "${arr[j]}" > "${arr[j+1]}" ]]
                then
                        temp="${arr[j]}"
                        arr[j]="${arr[j+1]}"
                        arr[j+1]="$temp"
                fi
        done
done

for item in "${arr[@]}"; do
        echo "$item"
done
```

Залючение

В результате выполнения работы был изучен сценарий bash, команды touch, find, case и shift, а также getopts. Был реализован вывод аргументов в обратном порядке, создание файла и установка прав доступа к нему, создан калькулятор для арифметических действий, вывод на экран заданного текста заданное количество раз с заданным промежутком времени, реализована сортировка в алфавитном порядке.